From Purchases to Communities

How User Activity Affects Product Network Construction

Tong Zhou, Sia Xie, Yilin Jiang, and Mingjiao Diao

STAT 236 Course Project | Fall 2026

Abstract

A user who purchases many products creates many links in a projected product network. Whether this helps community recovery depends on the user's preferences. We compare shared-purchaser counts, Jaccard similarity, inverse-activity weighting, and a normalized bipartite SVD baseline on 1,200 simulated datasets with known product categories. The design crosses activity heterogeneity, preference breadth, and purchase density while matching total purchases within each density setting. When active users retain category preferences, Count and Jaccard benefit from concentrated activity. At mean degree 12 and high heterogeneity, their mean misclustering rates are 13.04% and 13.15%, compared with 48.60% for activity weighting and 47.75% for normalized SVD. When the most active users have no category preference, recovery is poor. Weighting gives modest gains at medium heterogeneity but no clear gain at high heterogeneity. Exact category-count calculations and paired comparisons show why limiting user influence can discard useful evidence without restoring information absent from the remaining users. These findings concern the specified simulation model and do not establish a universal ranking of methods.

1 Introduction

User-product purchase records naturally form a bipartite network. Connecting products purchased by the same user creates a product network on which spectral clustering can identify communities. Such communities describe purchasing preferences; they need not correspond to functional similarity between products.

The construction is sensitive to user activity. A user purchasing d distinct products contributes d(d-1)/2 unordered product pairs. A small active group can therefore account for much of the projected network. This motivates downweighting active users, but activity alone does not determine information quality. A frequent buyer with a category preference can create useful within-category connections. A frequent buyer with no preference instead contributes category-independent connections in expectation.

We ask how activity heterogeneity and preference breadth interact with projection weights, when inverse-activity weighting improves on Count and Jaccard, and how these projections compare with a normalized SVD of the bipartite matrix. Our contribution is a controlled implementation and comparison of existing ideas, with known labels, matched purchase totals, exact moment calculations, and paired Monte Carlo uncertainty.

Newman (2001) motivates inverse group-size contributions in projected collaboration networks. Zhou et al. (2007) study resource allocation on bipartite networks, providing related projection context; our symmetric weighting is not their complete recommendation procedure. Dhillon (2001) motivates normalized bipartite spectral representations. Rohe et al. (2016) provide related spectral co-clustering ideas, while von Luxburg (2007) reviews spectral clustering. Our direct SVD baseline is inspired by this literature rather than an exact implementation of either original co-clustering algorithm.

2 Simulation design

Let B be the binary purchase matrix for n=1,000 users and p=300 products in K=3 equal categories. Product labels are fixed. Users independently receive a uniformly drawn preferred category, independently of activity, giving approximately balanced preference groups.

Each user samples d_u distinct products sequentially without replacement. Available preferred-category products have weight r_u; others have weight 1. The main ratio is r=2. This is a sampling weight ratio, not a fixed ratio of realized category counts. Products within a category have equal weights.

Table 1 specifies activity distributions with exactly matched purchase totals within each density setting. Heterogeneity changes how purchases are concentrated among users.

Mean purchases	Low heterogeneity	Medium heterogeneity	High heterogeneity
6	All users buy 6	80% buy 3; 20% buy 18	80% buy 1; 20% buy 26
12	All users buy 12	80% buy 6; 20% buy 36	80% buy 2; 20% buy 52

Table 1. Exact purchase-count distributions. Counts are randomly assigned to users independently of preferred category.

In the focused condition, r_u=2 for every user. In the broad-active condition, the most active 20% have r_u=1; others retain r_u=2. Independent random numbers break activity ties. With equal activity, a random 20% subset becomes broad.

The 12 conditions cross activity, preference setting, and density. Each has 100 independent datasets, with all methods evaluated on the same matrix. No real purchase records are used.

A separate pilot used 10 datasets per condition. Four conditions met the proposed difficulty criterion: median error between 0.05 and 0.40 for at least one method. Coverage and runtime checks passed, so we retained the initial design without selecting for method gaps. Main simulations use fresh seeds. Further calibration and unequal-popularity sensitivity analyses were not run.

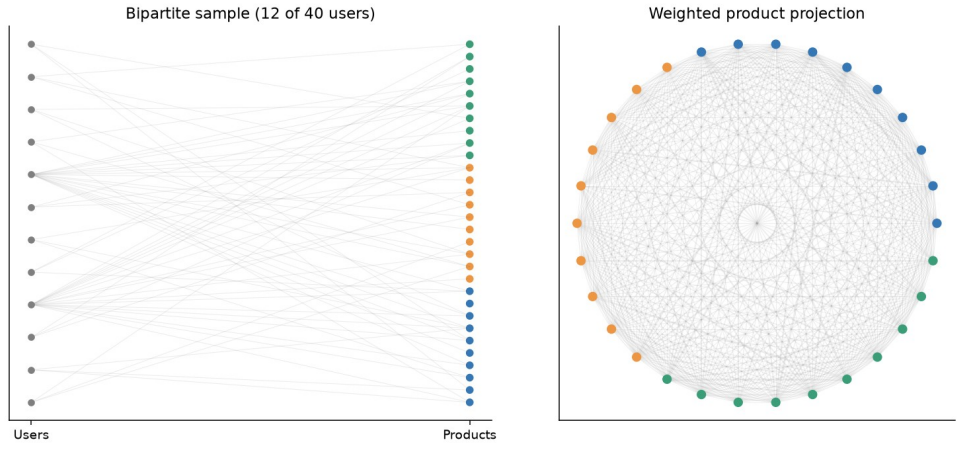


Figure 1. An illustrative small simulated network, separate from the main experiment. Colors indicate true product categories. The left panel shows 12 of 40 users; the right panel shows the weighted projection for all 40 users and 30 products.

3 Methods and theoretical calculations

For distinct products i and j, let S_ij=sum_u B_ui B_uj and b_i=sum_u B_ui. We construct three symmetric product networks:

$$W_{count}(i,j) = S_{ij}$$

$$W_J(i,j) = S_{ij} / (b_i + b_j - S_{ij})$$

$$W_{weighted}(i,j) = \text{sum over } u \text{ with } d_u >= 2 \text{ of } B_{ui} B_{uj} / (d_u - 1)$$

Jaccard is zero for a zero denominator. All projection diagonals are set to zero, and weighted edges are not thresholded. For each projection W, let D contain its row sums. We take the K largest algebraic eigenvectors of D^(-1/2) W D^(-1/2), normalize nonzero embedding rows, and apply k-means with 20 initializations and a recorded seed. Zero-strength products and zero embedding rows remain unassigned.

The direct baseline uses M=Du^(-1/2) B Dp^(-1/2), where Du and Dp contain user and product degrees. After removing zero-degree rows and columns, we retain the leading K right singular vectors, normalize their nonzero rows, and apply the same k-means settings. K is fixed at its true value for all methods. This baseline retains the implicit diagonal effect of M-transpose M, unlike the three projections.

Let s=p/K and C_uk denote purchases by user u from category k. Exchangeability within categories gives the conditional probability of purchasing a specified pair:

$$q_{same} = E[C_{uk}(C_{uk} - 1)] / [s(s - 1)]$$

$$q_{different} = E[C_{uk} C_{ul}] / s^2, \text{ for } k \text{ different from } l$$

Count expectations sum these probabilities over users; Weighted expectations divide each user's contribution by d_u-1 when d_u>=2. We calculate category-count moments by dynamic programming under the actual sequential sampling scheme. Reported block expectations also average over the independently uniform preferred categories. Appendix A gives the recurrence.

For r_u=1, the selected products form a uniform subset, so every distinct pair has inclusion probability d_u(d_u-1)/[p(p-1)]. Such a user adds no expected within-minus-between contrast. Exactly, a user contributes total unordered weight d_u(d_u-1)/2 under Count and d_u/2 under Weighted when d_u>=2; both contributions are zero for d_u<2. These identities establish quadratic versus linear total influence, not a clustering guarantee after normalization.

For Jaccard, we use the ratio of expected shared counts to expected union counts as an approximation. We check it against simulated means and do not identify it with the exact expected ratio. Raw contrasts have different units across projections, so their magnitudes alone cannot rank recovery quality.

4 Community recovery

The primary metric is misclustering rate after optimal permutation of cluster labels. We count unassigned products as errors and divide by all 300 products. Secondary metrics include ARI among assigned products, assignment coverage, edge contrasts, product-strength variation, and runtime. Each plotted confidence interval is a 95% Student-t Monte Carlo interval for a mean over 100 independent datasets. These are not prediction intervals for individual datasets.

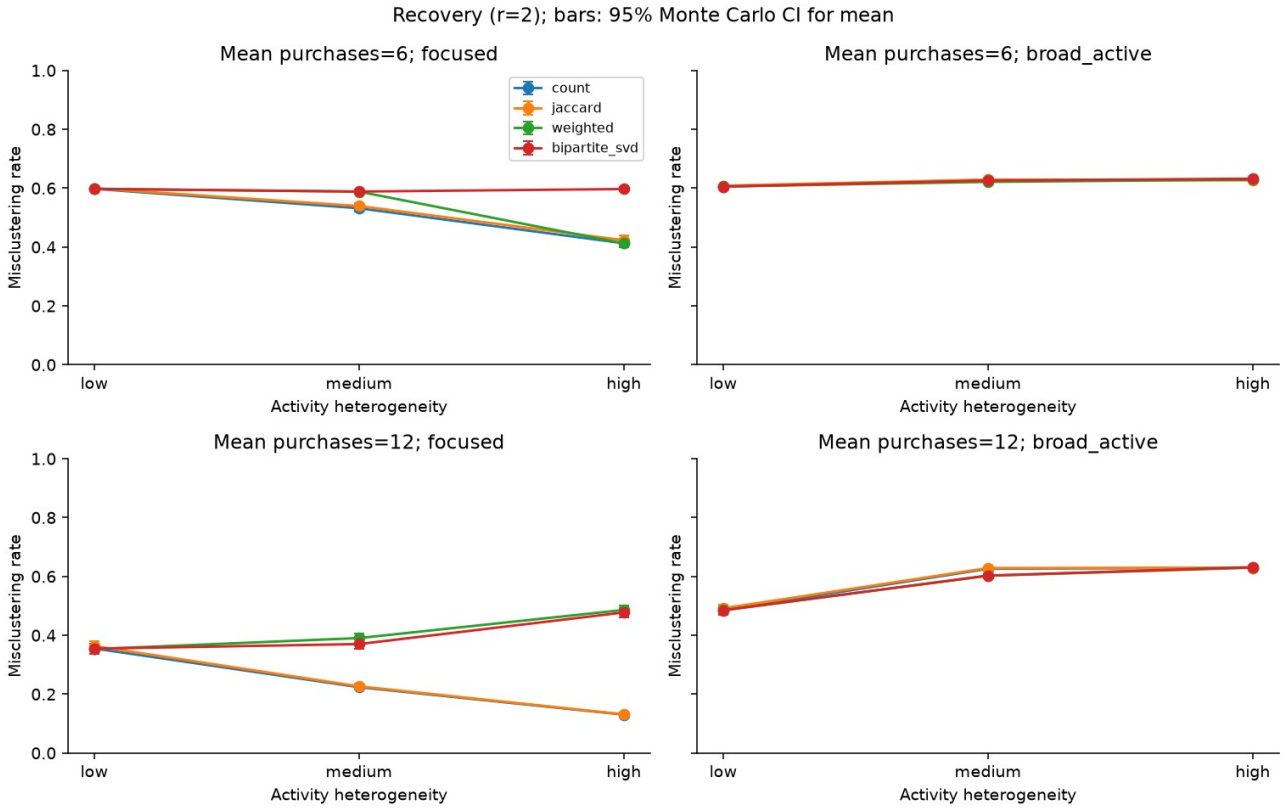


Figure 2. Mean misclustering rate across activity heterogeneity, with 95% Monte Carlo confidence intervals. Columns distinguish preference settings; rows distinguish mean purchase counts. Every point summarizes 100 datasets. Lower values indicate better recovery.

Focused active users substantially improve Count and Jaccard recovery. At mean purchase count 12, Count error decreases from 35.51% under low heterogeneity to 22.36% under medium and 13.04% under high heterogeneity. Jaccard follows closely at 36.33%, 22.65%, and 13.15%. Weighting instead gives 35.31%, 39.05%, and 48.60%; SVD gives 35.46%, 37.01%, and 47.75%.

The sparse focused setting shows a different weighting trajectory: at mean 6, Count improves from 59.77% to 53.17% to 41.28%, whereas Weighted gives 59.76%, 58.81%, and 41.23%. Thus, even within focused preferences, the effect of heterogeneity is not uniformly worsening for Weighted.

Broad-active conditions have substantially poorer recovery. At mean 12, Count errors are 48.53%, 62.60%, and 62.94% across heterogeneity levels. At mean 6, they are 60.54%, 62.83%, and 62.87%. Medium and high heterogeneity both yield weak recovery; the exact ordering is not identical for every method. All methods assign every product, and no main-experiment product is isolated. Poor recovery therefore reflects incorrect assignments rather than missing coverage.

5 Paired effects of activity weighting

Paired differences isolate the method comparison within each generated dataset. Define the difference as Count error minus Weighted error. Positive values favor Weighted; negative values favor Count. Table 2 reports differences in percentage points for mean purchase count 12.

Preferences	Activity	Count minus Weighted	95% Monte Carlo interval
Focused	Low	0.21	0.01 to 0.41
Focused	Medium	-16.69	-18.19 to -15.18
Focused	High	-35.57	-36.99 to -34.14
Broad active	Low	-0.01	-0.06 to 0.04
Broad active	Medium	2.36	1.77 to 2.96
Broad active	High	-0.04	-0.49 to 0.42

Table 2. Paired mean error differences over 100 datasets per condition. Intervals are unadjusted for multiple comparisons; statistical detectability does not imply a large practical effect.

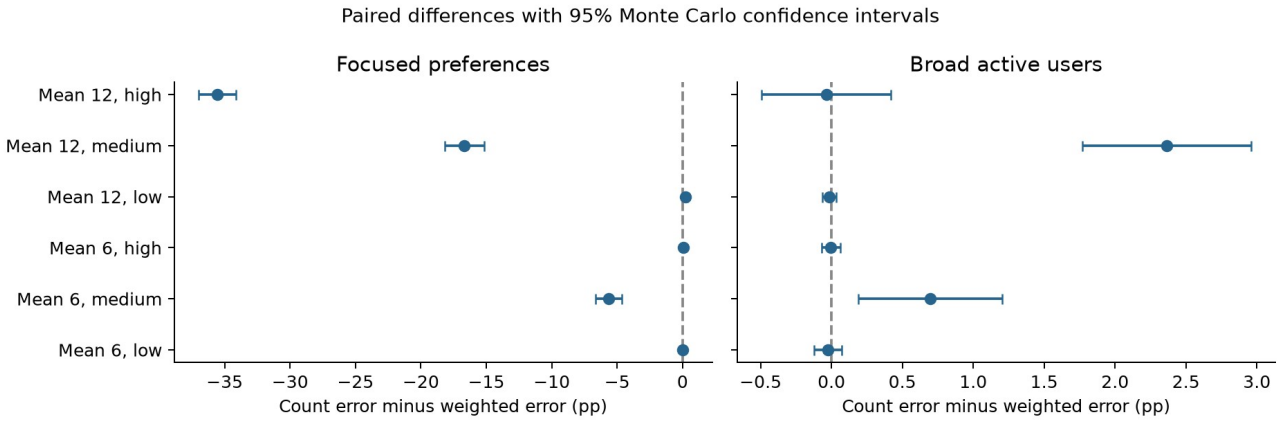


Figure 3. Paired Count-minus-Weighted differences for both purchase densities. Zero marks equal mean error. Negative values favor Count.

Weighting helps most clearly at medium heterogeneity with broad active users: the gain is 2.36 percentage points at mean 12 and 0.70 points at mean 6, with intervals excluding zero. These improvements leave errors near 60%, so they do not indicate successful category recovery. At high heterogeneity, the differences are small and intervals include zero. At low heterogeneity, Count and Weighted matrices differ only by a common scalar and have the same normalized spectral matrix in exact arithmetic. Small measured discrepancies can arise from numerical and k-means initialization differences; they are not evidence for a structural weighting advantage.

The interpretation is that activity weighting trades off suppression of category-independent pair contributions against retention of useful repeated evidence. This tradeoff is unfavorable when high-degree users remain focused. The observed gains in broad-active conditions are limited by the information supplied by the remaining users.

6 Diagnostics and comparison with direct SVD

At mean 6 with high heterogeneity, the 800 preference-bearing users each purchase one product in the broad-active condition. They create no product pairs. The 200 users purchasing 26 products have r_u=1, giving zero expected category contrast. Count and Weighted therefore have exactly zero population block contrast. This is an information limitation for the pair projections, and explains why reducing active-user weights cannot create the missing contrast.

At mean 12 with high heterogeneity, focused users each purchase two products while broad users each purchase 52. Useful pair information remains, but it is sparse. A broad user's total Count contribution is 1,326 unordered pair units, compared with one unit for a focused user. Weighted contributions become 26 and one. The corresponding broad-user shares are about 99.70% and 86.67%. Weighting reduces their dominance, but they still supply most projected pair mass. Pair mass is a mechanism diagnostic, not a substitute for spectral signal-to-noise analysis.

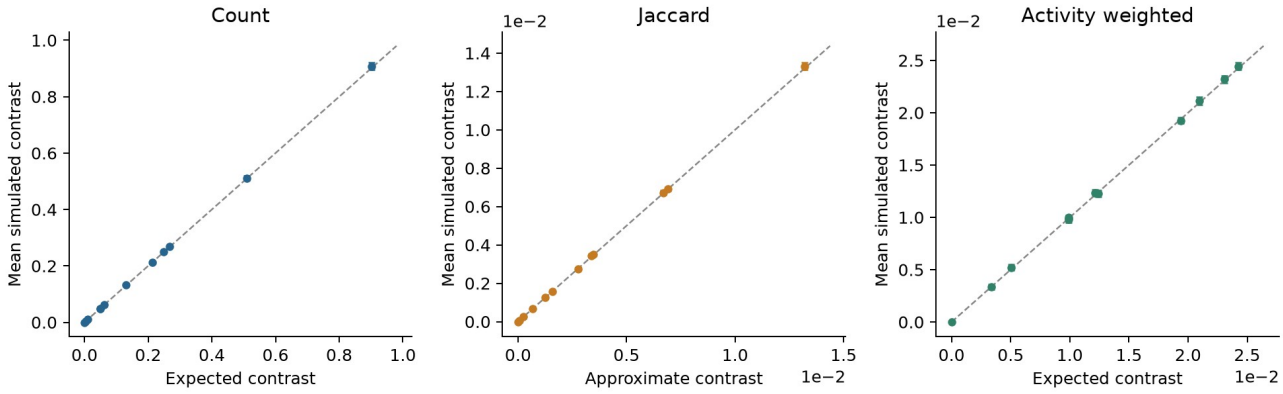


Figure 4. Simulated mean within-minus-between edge weights versus theoretical values, with dataset-level Monte Carlo intervals. Each panel uses its own units. The dashed line indicates agreement. Jaccard uses an approximate expected contrast.

Count and Weighted simulated contrasts track the theoretical calculations. For example, at mean 12 with focused high heterogeneity, Count's theoretical contrast is 0.9020 and its simulated mean is 0.9075; Weighted's corresponding values are 0.02098 and 0.02116. Across conditions, the relative discrepancy between simulated mean Jaccard within or between weights and the ratio approximation is below 0.23%. Agreement here supports the approximation in this design, without making it an exact identity.

Across conditions, mean product-strength coefficients of variation are 0.199 for Count, 0.125 for Jaccard, and 0.187 for Weighted. Lower strength variation alone does not ensure better recovery. Count and Jaccard have mean edge-rank correlation 0.9746 across datasets. Their largest condition-level mean error gap is 1.03 percentage points. Nevertheless, their mean clustering agreement ARI is 0.7012, so similar average recovery does not imply identical partitions. Equal expected product popularity helps explain the similar edge rankings; unequal popularity remains untested.

SVD performs similarly to Weighted in several conditions. In the focused high-heterogeneity setting at mean 12, SVD error exceeds Count by 34.71 percentage points, with paired 95% interval 33.22 to 36.20. At mean 6 in the same preference/activity setting, SVD error is 59.70%, while Count and Weighted are near 41%. No general SVD advantage is observed.

The identity M-transpose M=Dp^(-1/2) B-transpose Du^(-1) B Dp^(-1/2) connects SVD to a weighted Gram matrix. It clarifies their related normalization choices, but different user weights and diagonal treatments prevent attributing the entire performance gap to one operation.

7 Discussion and reproducibility

The answer to our first question is that activity heterogeneity changes the leverage of whichever preference behavior is concentrated in active users. With focused users, concentrated activity strengthens Count and Jaccard recovery. With broad active users, it shifts purchases toward users lacking category information, and recovery becomes poor. Matching total purchases controls the data volume, but it does not preserve the amount or arrangement of informative pair evidence.

For the second question, activity weighting provides modest improvements over Count under medium heterogeneity with broad active users, but little benefit at high heterogeneity. It can substantially hurt when active users are focused. Jaccard and Count behave similarly in this balanced-popularity design. For the third question, the normalized bipartite SVD baseline offers no universal improvement and can lose much of the recovery advantage obtained by Count. These findings apply to the specified baseline and do not imply that all SVD or co-clustering procedures perform poorly.

Several limits constrain interpretation. Projected edges are dependent because one user creates many edges. We do not use an independent-edge stochastic block model or claim a universal recovery threshold. Categories are equal in size, within-category product weights are equal, preferences are single-category, K is known, and only r=2 is studied in the main experiment. We have no real purchase-data validation. Broad-active high heterogeneity simultaneously weakens the information supplied by less active users; it is not a pure test of removing nuisance connections while holding informative evidence fixed. Confidence intervals measure Monte Carlo precision under this model, and comparisons are not adjusted for multiplicity.

A useful extension would hold the informative users' purchases fixed while adding or varying broad active users, separating nuisance-background effects from loss of informative observations. Other extensions include unequal product popularity, mixed preferences, varying r, and controlled ablations of user normalization and diagonal removal. Such experiments should be labeled as additional analyses rather than selected to produce a preferred ranking.

The implementation records dataset and k-means seeds, exact activity distributions, package versions, and atomic dataset checkpoints. Pilot and main root seeds are 23601 and 23603, with no overlapping recorded dataset seeds. Python 3.11.14 and all numerical package versions are recorded in the manifests and requirements lockfile. The generator uses NumPy weighted sampling without replacement; spectral extraction uses the largest algebraic eigenvalues; k-means uses 20 starts. Main results contain 1,200 datasets and 4,800 method rows. Five tests check design identities, label matching and zero handling, known partition recovery, uniform-sampling formulas, and theory against simulation.

Across all conditions, mean method runtimes are approximately 12.5 ms for Count, 12.8 ms for Jaccard, 14.8 ms for Weighted, and 48.2 ms for SVD. Timings include each method's construction or decomposition and clustering, but exclude generation, theoretical calculations, and diagnostics. They were collected with four worker processes and one numerical-library thread per worker, so shared-machine load limits timing generalization.

The accompanying code and experiment tables reproduce the analysis. Raw B matrices are regenerated from recorded seeds rather than stored individually. The principal conclusion is conditional: user activity should be interpreted together with preference information. Automatically suppressing active users can discard the strongest available evidence, while suppression alone cannot recover category information absent from the rest of the data.

References

Dhillon, I. S. (2001). Co-clustering documents and words using bipartite spectral graph partitioning. Proceedings of the Seventh ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 269-274. https://doi.org/10.1145/502512.502550

Newman, M. E. J. (2001). Scientific collaboration networks. II. Shortest paths, weighted networks, and centrality. Physical Review E, 64, 016132. https://doi.org/10.1103/PhysRevE.64.016132

Rohe, K., Qin, T., and Yu, B. (2016). Co-clustering directed graphs to discover asymmetries and directional communities. Proceedings of the National Academy of Sciences, 113(45), 12679-12684. https://doi.org/10.1073/pnas.1525793113

von Luxburg, U. (2007). A tutorial on spectral clustering. Statistics and Computing, 17, 395-416. https://doi.org/10.1007/s11222-007-9033-z

Zhou, T., Ren, J., Medo, M., and Zhang, Y.-C. (2007). Bipartite network projection and personal recommendation. Physical Review E, 76, 046115. https://doi.org/10.1103/PhysRevE.76.046115

Reproducing the analysis

From the project directory, create a Python 3.11 environment and install requirements-lock.txt. The following commands use the frozen initial design and saved checkpoints.

```
python -m unittest discover -s tests -v
python run_experiments.py --stage main --frozen-config frozen_config.json \
--workers 4 --resume
python run_experiments.py --stage main --analyze-only
python scripts/build_report_assets.py
```

For a fresh independent run with the same design, add a new --seed and --output directory and omit --resume. The delivered outputs/main directory contains results.csv, diagnostics.csv, summary.csv, paired_differences.csv, manifest.json, and dataset checkpoints. The experiment manifest is the authoritative record of the completed main design.

Appendix A Exact category count recurrence

For a user with d purchases and preference ratio r, let P_t(c) be the probability of selecting c preferred-category products after t draws. Let s=p/K and h=p-s. Set P_0(0)=1. Conditional on state c, the next product is preferred with probability

$$a_t(c) = r(s-c) / [r(s-c) + h-(t-c)]$$

The updates are P_(t+1)(c+1) += P_t(c) a_t(c) and P_(t+1)(c) += P_t(c)[1-a_t(c)]. Only feasible states are visited. At t=d, let C denote the preferred count and R=d-C. Conditional on C, nonpreferred products are a uniform R-subset of h products. For any one nonpreferred category, its ordered-pair moment is E[R(R-1)] s(s-1)/[h(h-1)]. Thus, defining

$$A = E[C(C-1)] + (K-1) E[R(R-1)] s(s-1) / [h(h-1)]$$

and averaging over a uniform preferred category gives q_in=A/[K s(s-1)] and q_out=[d(d-1)-A]/[K(K-1)s squared]. Summing user contributions gives the Count and Weighted expectations. This averages over preferred-category assignments; it is not the expectation conditional on a particular realized assignment.

Appendix B Complete mean misclustering rates

All entries below are percentages, based on 100 datasets per condition. Broad active means that the most active 20% have r=1. Full precision, standard errors, and intervals are in the accompanying CSV files.

Mean	Activity	Preferences	Count	Jaccard	Weighted	SVD
6	Low	Focused	59.77	59.94	59.76	59.75
6	Low	Broad active	60.54	60.83	60.56	60.54
6	Medium	Focused	53.17	53.92	58.81	58.84
6	Medium	Broad active	62.83	62.91	62.13	62.60
6	High	Focused	41.28	42.31	41.23	59.70
6	High	Broad active	62.87	62.64	62.87	63.21
12	Low	Focused	35.51	36.33	35.31	35.46
12	Low	Broad active	48.53	49.17	48.55	48.48
12	Medium	Focused	22.36	22.65	39.05	37.01
12	Medium	Broad active	62.60	62.84	60.23	60.28
12	High	Focused	13.04	13.15	48.60	47.75
12	High	Broad active	62.94	62.94	62.97	63.16